

# DirectSpeech2LLM: A Simple End-to-End Framework to Mitigate Prompt Overfitting in Speech-LLMs

**Hemant Yadav\***  
IIIT Delhi, India  
hemantya@iiitd.ac.in

**Sunayana Sitaram**  
Microsoft Research, India  
sunayana.sitaram@microsoft.com

**Roger Zimmermann**  
National University of Singapore  
dcsrz@nus.edu.sg

**Rajiv Ratn Shah**  
IIIT Delhi, India  
rajivrtn@iiitd.ac.in

## Abstract

Speech-LLMs often exhibit prompt overfitting, where models solely trained on automatic speech recognition (ASR) instruction fail to generalize to new instructions such as speech translation and continue to behave primarily as ASR system. We propose **DirectSpeech2LLM**, a simple end-to-end framework that preserves the instruction-following ability of the LLM on unseen tasks when conditioned on speech. It computes distance-based CTC loss over the frozen LLM embedding matrix and uses greedy CTC labels to derive geometrically and temporally aligned speech embeddings respectively as an input to the LLM. Trained solely on 960 hours of LibriSpeech ASR data, DirectSpeech2LLM outperforms the cascaded system on ASR (seen task) and generalizes zero-shot to speech translation and emotion recognition (two unseen tasks), closely matching the cascaded system upper bound on these two new instructions despite seeing neither during training. We also find that geometric alignment strength plays a smaller role than previously assumed, as our modified CTC loss is shown to provide sufficient implicit geometric grounding without requiring an explicit regression loss. Results are consistent across two LLM families and scale with both more training data and model capacity.

## 1 Introduction

Large language models (LLMs) have shown a remarkable ability to learn rich linguistic information from raw text, enabling strong generalization across language understanding, generation, and reasoning tasks [1, 2, 3]. Speech, in contrast, is a continuous acoustic signal that conveys the same underlying linguistic information that LLMs already represent effectively in discrete textual form. The ability of a pretrained LLM (the “brain”) to process auditory inputs (the “ears”) while preserving its original generalization ability is referred to as a desirable Speech-LLM [4]. Speech also encodes other information such as speaker identity, prosody, and noise [5]. However, current large audio language models are shown to behave largely as transcription systems, relying on lexical semantics and underutilizing acoustic cues [6]. This work focuses on improving the linguistic modeling of Speech-LLMs to new tasks not seen during training and leave improving non-linguistic modeling for future.

Enabling LLMs to process linguistic content present in speech requires transforming speech into embeddings aligned with the LLM input embedding space (IES) along two dimensions: geometric

\*indicates the corresponding author.

Code available at: <https://github.com/raotnameh/DirectSpeech2LLM>

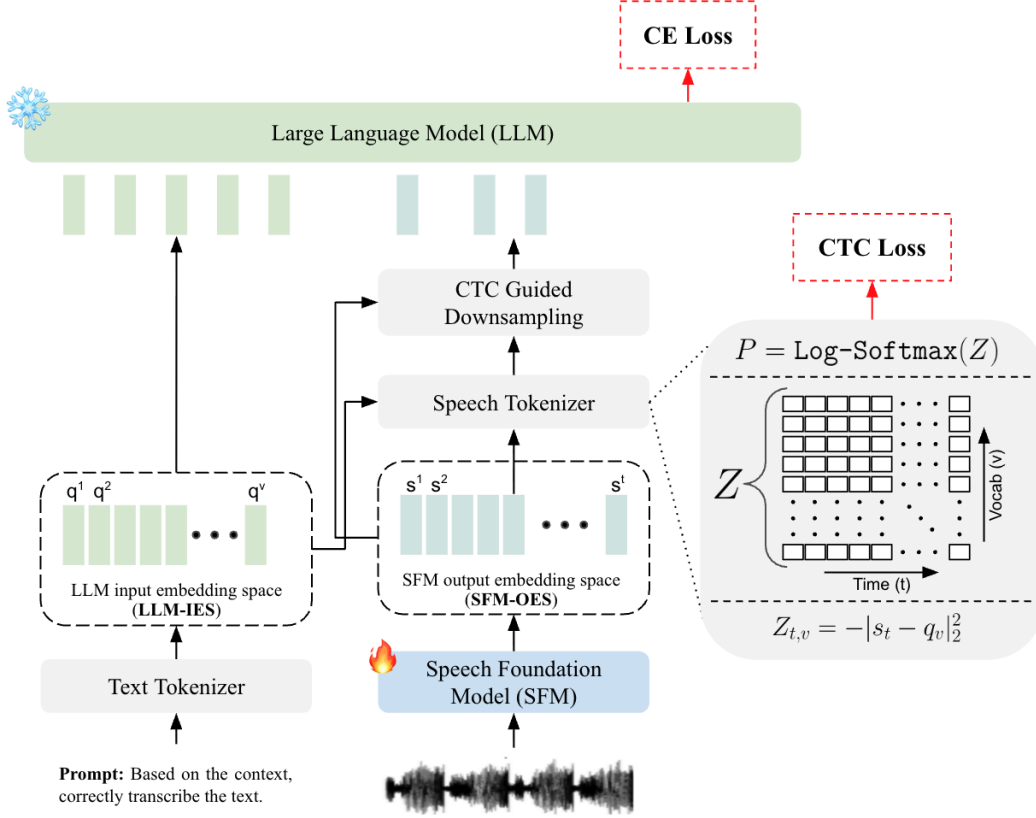


Figure 1: Overview of **DirectSpeech2LLM**. Given an input speech signal, the SFM produces frame representations  $s_t \in \mathbb{R}^d$ , where  $d$  matches the dimensionality of the LLM input embeddings  $Q$ . Logits are computed as negative squared distances ( $Z_{t,v}$ ) followed by log-softmax and CTC loss ( $\mathcal{L}_{\text{ctc}}$ ) is applied. Next, the resulting greedy CTC predicted labels are then used to downsample SFM output embeddings via mean pooling. The LLM is fed with the resulting updated sequence embeddings from SFM and is trained with cross-entropy loss ( $\mathcal{L}_{\text{llm}}$ ). The reader should keep in mind that the LLM-IES serves only as a geometric anchor for the SFM-OES, without being replaced or discretized with the LLM input embeddings. Commitment loss is not shown in the figure for clarity.

and temporal [7]. Geometric alignment ensures that speech derived embeddings are consistent with the geometry of the trained LLM-IES by minimizing a distance metric. Temporal alignment ensures that the embedding sequence exhibits statistical properties consistent with those observed during LLM training.

Existing Speech-LLMs connect a pretrained speech foundation model (SFM) to an LLM using a projection module<sup>2</sup>, implemented using a linear layer [8, 9], non-linear layers [10], CNN layers [11], Q-former [12] or a combination of them. While effective on seen tasks, current models often fail to generalize to unseen tasks [13] or prompt overfitting<sup>3</sup>. For example, a Speech-LLM model trained solely on ASR task continues to behave as an ASR system even when prompted differently to perform translation or question answering [4, 14].

We hypothesize that this lack of generalization arises because of two factors. First, the SFM output embedding space (OES) is not constrained geometrically with the LLM-IES, so embeddings can occupy arbitrary regions in  $\mathbb{R}^d$  during training, degrading the LLM’s generalization ability. Prior work has shown that LLMs do tolerate small perturbations in their input embeddings, but performance degrades sharply beyond a threshold [15]. Since the LLM is frozen, only the SFM outputs can be

<sup>2</sup>for simplicity, from now on we assume the projection module is also part of SFM.

<sup>3</sup>The terms *prompt overfitting* and *failure to generalize to unseen tasks* are used interchangeably to describe the same phenomenon. Both refer to the loss of the original instruction-following ability of the underlying LLM in Speech-LLMs

constrained to lie close to the LLM-IES, otherwise the system reduces to a task-specific pipeline similar to Whisper [16].

Second, most approaches rely on fixed temporal downsampling [17, 18, 19], which is inherently misaligned with the highly variable temporal dynamics of continuous speech. Because phoneme durations vary across speakers, speaking rates, and linguistic contexts, fixed downsampling produces inconsistent sequence boundaries for identical linguistic units causing overfitting. Consequently, the embedding sequence length for a given token becomes an artifact of speaking rate rather than semantic content. Given the LLM is fixed, such temporal mismatch shifts the input distribution away from that seen during LLM’s training, impacting its generalization ability [20].

In this work, we propose **DirectSpeech2LLM**, a simple end-to-end Speech-LLM framework that jointly enforces geometric and temporal alignment. Figure 1 illustrates the overall framework. To save compute time DirectSpeech2LLM is trained in two stages to save on compute time. In Stage 1, the SFM is trained using a modified CTC objective augmented with vector quantization (VQ) to enforce alignment along the two dimensions of LLM. In Stage 2 (end-to-end), we add the LLM cross-entropy loss to the Stage 1 objectives to further improve performance.

AlignFormer enforces temporal alignment by dynamically downsampling the speech embeddings using Connectionist Temporal Classification (CTC) to match the LLM’s input temporal distribution [13]. However, they do not enforce geometric constraint between the SFM-OES and LLM-IES. A similar work, Wav2Prompt [7] uses continuous integrate-and-fire (CIF) module to align temporally and regression loss to map speech embeddings to equivalent LLM tokens but suffers from a train/inference mismatch due to length dependent weight scaling and the performance is very sensitive to geometric alignment strength on unseen tasks, unlike our method.

Empirically, DirectSpeech2LLM achieves strong performance on ASR (seen task) and generalizes zero-shot to speech translation and emotion recognition (two unseen tasks or two new instructions). DirectSpeech2LLM also demonstrates robustness to out-of-domain setting on the ASR task, similar to CTC trained methods, which is shown to be a limitation of simple adapter based methods [21, 22].

Our main contributions are as follows.

1. We propose DirectSpeech2LLM, a simple end-to-end Speech-LLM framework that preserves LLM’s generalization ability to new instructions without relying on (i) large amount of multitask instruction tuning dataset or (ii) costly multiple forward passes to LLM.
2. Our method treats CTC as a primary objective rather than an auxiliary loss, giving the SFM three independently usable operating modes at test time: (i) standalone CTC-based ASR, (ii) CTC + n-gram LM ASR, or (iii) full end-to-end Speech-LLM. Unlike most prior approaches that tightly couple the SFM and LLM into a single pipeline, SFM output embeddings here are fed *directly* to the LLM without any projection module, discretization, or separate adapter – hence the name **DirectSpeech2LLM**.

The rest of the paper is organized as follows: Section 2 reviews related work, Section 3 describes the proposed method, Section 4 details the experimental setup, Section 5 presents results and ablations, and Section 6 concludes with future directions and limitations.

## 2 Related works

Recent advancements in Speech-LLMs have explored various strategies to align SFM with LLM. One common direction focuses on solving the modality gap through behavior alignment. BLSP [23] trains Speech-LLMs to mimic LLM responses to repeat task based prompts. While this improves zero-shot capabilities, performance lags behind instruction-tuned models. SALAD [14] extends this work by combining cross-modal distillation with targeted synthetic data to improve alignment. But behavior alignment methods require two LLM forward passes during training, roughly doubling compute, compared to one forward pass in instruction tuned methods. Other approaches rely on computationally expensive large-scale synthetic or data-driven pretraining to preserve LLM generalization capabilities [24, 12, 11, 25]. Behavior alignment is applied on the LLM-OES while our method is aligning the LLM-IES. Therefore, these two are complementary and can be applied together.

Another direction, focuses on temporal alignment using CTC. In the context of Speech-LLMs AlignFormer [13] utilizes CTC alignments as a temporal guide to form dynamic windows for a Q-Former adapter. A similar work is Soundwave [25] which uses simple projection layers instead of Q-former. LegoSLM [26] bypass adapters completely by utilizing CTC posterior distributions and finetune the LLM. It computes a weighted sum of the LLM’s existing text embeddings based on CTC posteriors to create pseudo-speech embeddings. While TASU [27] uses simulated pseudo CTC erroneous text during training to learn the adapter without using paired speech data.

Prior speech translation work [28, 29] separates temporal alignment (CTC) and distribution-level geometric alignment (Optimal Transport). DirectSpeech2LLM unifies both by computing CTC logits as negative squared distances to frozen LLM embeddings as fixed codebooks, jointly achieving dynamic temporal compression and geometric grounding.

Unlike Wav2Prompt [7], which uses CIF followed by regression, DirectSpeech2LLM uses modified CTC to compute logits as negative squared distances to the frozen LLM embedding matrix. As CTC is the canonical ASR framework, DirectSpeech2LLM provides a conceptually simpler and robust, single objective solution that unifies temporal and geometric alignment without extra hyper parameters or mismatch between train/inference. Wav2Prompt also collapses if regression loss (geometric) is removed compared to our method where commitment loss (geometric) is optional as CTC implicit has geometric constraint, making our method simpler. Lastly the geometric loss used in Wav2Prompt does not take into account negative LLM token embeddings, whereas our distance-based logits i.e., move the positive embedding closer while negative embeddings are pushed farther apart. This could be the reason for our method’s less sensitivity to the regression loss

Different to AlignFormer [13], which uses CTC to handle temporal alignment but leaves the SFM-OES geometrically unconstrained. DirectSpeech2LLM enforces geometric grounding using distance-based logits (implicit) and a commitment constraint (explicit). Which is shown to prevent prompt overfitting without relying on input ordering tricks as in Alignformer. Again, AlignFormer input ordering trick is complementary to our method both can be applied together.

### 3 Method

#### 3.1 Preliminaries

Please refer to Section A.1 for preliminaries.

#### 3.2 Problem formulation

Let  $X = (x_1, \dots, x_M)$  denote an input acoustic sequence and  $Y = (y_1, \dots, y_U)$  its corresponding ground truth labels, where  $y_u \in \mathcal{V}_{\text{ctc}} \subseteq \mathcal{V}_{\text{llm}}$ . We assume a trained LLM ( $\mathcal{M}_\theta$ ) with frozen parameters  $\theta$  and embedding matrix  $Q \in \mathbb{R}^{|\mathcal{V}_{\text{llm}}| \times d_{\text{llm}}}$ . where each row  $q_j$  uniquely corresponds to one of the LLM’s vocabulary embedding. The SFM produces frame level embeddings with the same dimension as  $Q$ .

$$E_\phi : X \mapsto S = (s_1, \dots, s_T), \quad s_t \in \mathbb{R}^{d_{\text{llm}}},$$

Our objective is to learn  $\phi$  such that  $S$  is aligned with LLM-IES i.e., each  $s_t$  lies closer to its equivalent in  $Q$  in both temporal and geometric dimensions.

#### 3.3 Speech tokenizer

Speech Tokenizer computes frame level logits given (i) SFM output embeddings and (ii) LLM input embedding matrix  $Q$  using negative squared Euclidean distance similar to [30].

$$Z_{t,v} = -|s_t - q_v|_2^2. \quad (1)$$

Expanding,

$$Z_{t,v} = -(\|s_t\|_2^2 - 2s_t^\top q_v + \|q_v\|_2^2). \quad (2)$$

Since  $\|s_t\|_2^2$  does not depend on  $v$ , this formulation is equivalent to a linear projection with fixed weights  $Q$ . This is equivalent to selecting the nearest token embedding in the LLM input embedding space.

The LLM has a large vocabulary  $\mathcal{V}_{\text{llm}}$  in thousands if not hundreds of thousands. Directly using this vocabulary for CTC loss is suboptimal. CTC training is more stable and sample efficient when prediction units correspond to smaller, frequently occurring acoustic segments.

To obtain acoustically suitable prediction units, we instead construct a dedicated CTC vocabulary  $\mathcal{V}_{\text{ctc}}$  using byte-pair encoding (BPE) trained on the normalized LibriSpeech transcripts<sup>4</sup>. We use a vocabulary size of 1000, which is a widely adopted starting point in end-to-end ASR systems, particularly for CTC and seq2seq baselines on the LibriSpeech dataset, as it provides a favorable trade-off between sequence length and acoustic consistency<sup>5</sup>. In the next step, we discard tokens which do not appear in the Qwen-3 LLM tokenizer vocabulary, resulting in 984 valid tokens for computing CTC loss (see Algorithm 2 in Section A.7 for pseudo code). For experiments with Phi-3, we initialize BPE with 2000 merge operations to obtain a final retained subset comparable in size to that derived from Qwen-3. The reader should keep in mind that the LLM still generate/output tokens over its original full vocabulary space.

During training, frame level logits are computed over the full LLM embedding matrix  $Q$  followed by log-softmax, meaning the negatives in the softmax function are consisting of the full LLM vocabulary. This is different from Wav2Prompt. Our earlier experiments showed that it resulted in better geometric alignment during stage 1. The final CTC loss is calculated only over  $\mathcal{V}_{\text{ctc}}$  vocabulary indices. We obtain:  $P_{\text{ctc}} = \text{logsoftmax}(Z[\mathcal{V}_{\text{ctc}}])$ . The CTC objective is then:

$$\mathcal{L}_{\text{ctc}} = -\log \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^T P_{\text{ctc}}(\pi_t | t), \quad (3)$$

where  $\mathcal{B}$  denotes the CTC collapse function. The modified CTC loss enforces explicit temporal alignment and implicit geometric alignment between the SFM-OES and the LLM-IES.

### 3.4 CTC guided downsampling

For each output frame of SFM, we compute a greedy label assignment:

$$\hat{I}_t = \arg \min_j |s_t - q_j|_2^2. \quad (4)$$

Next, we partition the output embeddings into consecutive runs of identical predictions. Let  $\mathcal{T}_i$  denote the set of indices belonging to the  $i$ -th run. Then each segment embedding is obtained by mean pooling the corresponding identical labels.

$$s_i^{\text{seg}} = \frac{1}{|\mathcal{T}_i|} \sum_{t \in \mathcal{T}_i} s_t, \quad i = 1, \dots, U'.$$

Collecting all segment embeddings yields  $S_{\text{seg}} \in \mathbb{R}^{U' \times d_{\text{llm}}}$ , where  $U'$  is the number of runs.

Lastly we global mean pool all the blank segments into 1 embedding. This step compresses the original SFM output embedding sequence of length  $T$  into 1 blank embedding, followed by remaining non-blank embeddings. The position of blank embedding is always at the start and the equivalent token in  $Q$  is underscore ( $\_$ ).

Exploring learnt embeddings for blank token and its ability to capture paralinguistic information of the speech utterance is out of scope of this work and is left for future work.

### 3.5 Commitment loss

Using the greedy CTC labels from Equation 4, we explicitly constrain SFM outputs to be close to the pretrained LLM-IES using a commitment loss:

$$\mathcal{L}_{\text{commit}} = \frac{1}{\sqrt{d_{\text{llm}}}} \sum_{t=1}^T M_t |s_t - q_{\hat{I}_t}|_2^2. \quad (5)$$

<sup>4</sup><https://openslr.trmal.net/resources/11/librispeech-lm-norm.txt.gz>

<sup>5</sup><https://speechbrain.readthedocs.io/en/stable/tutorials/tasks/speech-recognition-from-scratch.html>

Where  $M_t = \mathbb{1}[\hat{I}_t \neq b]$  is the mask of valid indices excluding blank token  $b$ . Commitment loss is *not* applied to the blank token indices because it does not encode linguistic information of the input utterance that has an equivalent in  $Q$ . Blank embedding is similar to Prefix-Tuning in LLM [31], that is it is same for all the samples in training.

The reader should keep in mind that the modified CTC loss (in Section 3.3) itself encourages the SFM output to lie closer to the LLM-IES geometrically, not as directly as using a commitment loss.

### 3.6 LLM

The segmented embeddings ( $S_{\text{seg}}$ ) are concatenated with prompt embeddings  $E_p$  (e.g., “Prompt: Correctly transcribe the following text:”). The resulting interleaved sequence is fed to the LLM and is trained using a standard auto regressive cross-entropy loss over the target text sequence  $Y = (y_1, \dots, y_U)$ .

$$\mathcal{L}_{\text{llm}} = - \sum_{u=1}^U \log p_{\theta}(y_u \mid X = [E_p = Q(\text{Prompt}), S_{\text{seg}}], y_{<u}).$$

### 3.7 Overall training objective

The final loss is:

$$\mathcal{L} = \mathcal{L}_{\text{ctc}} + \mathcal{L}_{\text{commit}} + \lambda_{\text{llm}} \mathcal{L}_{\text{llm}}. \quad (6)$$

The commitment loss is clamped to never exceed CTC loss because its targets are derived from greedy CTC predictions, meaning commitment loss is logically downstream of CTC. If CTC predictions are poor, the greedy labels are noisy, and the commitment loss would be pushing SFM embeddings toward incorrect geometric anchors in the LLM-IES.

Only the SFM parameters,  $\phi$ , are optimized and the LLM parameters are fixed. Training proceeds in two stages. In stage 1 we set ( $\lambda_{\text{llm}} = 0$ ). In the stage 2 (end-to-end) the LLM loss is enabled, allowing contextual feedback from the frozen LLM to further improve performance. For full training pseudocode see Algorithm 1 in Section A.7.

## 4 Experimental details

### 4.1 Training data

All models are trained using only the English ASR data (single task training setup) with SpecAugment [32].

**LibriSpeech:** We train on the full 960-hour LibriSpeech corpus [33]. This is referred as 1k data.

**Common Voice:** We additionally experiment with training on combined 1800 hours of the English subset of Mozilla Common Voice (version 24) [34] and 960 hours LibriSpeech. We use the official train split and no additional filtering or data cleaning is applied. This is referred as 2.8k data.

### 4.2 Model configuration

**LLM Backbone:** We use Qwen3-4B<sup>6</sup> and Phi3.1-3.8B<sup>7</sup> from HuggingFace as the language model backbone. All parameters of the LLM, including its token embedding matrix  $Q$ , remain frozen during training.

**SFM backbone:** It is initialized from the ASR finetuned Data2Vec-Large [35] checkpoint having 24 transformer layers ( $\approx 300$  million parameters). The original linear CTC head is removed and reinitialized to match the LLM input embedding dimension and is retrained with the modified CTC loss during stage 1.

<sup>6</sup><https://huggingface.co/Qwen/Qwen3-4B>

<sup>7</sup><https://huggingface.co/microsoft/Phi-3-mini-128k-instruct>

### 4.3 Training procedure

Training proceeds in two stages. Stage 1 is computationally cheap to temporally align SFM-OES and act as a good starting point for stage 2 training to save on compute. All training runs use only English ASR data, no other task specific supervised finetuning is done in any way, unless mentioned otherwise. All models are trained for a fixed number of optimization steps, rather than until convergence. Specifically, Stage 1 is trained for 50k steps, while Stage 2 is trained for 20k steps, unless mentioned otherwise. For more training details, please refer to Section A.2.

### 4.4 Evaluation procedure

During training both the blank embeddings and non blank embedding were fed to the LLM (WB setup). During inference for the ASR task, we follow the same setup. But on unseen tasks, we find that not using blank tokens (NB setup) consistently gives better results. Therefore we do not use blank token when evaluating on unseen tasks (NB setup). We use different task prompts for each task as shown in Figure 6 in the Section A.8. We use mms text normalization<sup>8</sup> to normalize the LLM output and ground truth.

**Automatic Speech Recognition (ASR):** We evaluate in-domain ASR performance on the LibriSpeech test-clean and test-other splits and out-of-domain (OOD) using VoxPopuli (VP) test set [36]. The number of LLM output words are clamped at  $\max(\text{CTC} + 5, \lfloor 1.1 \times \text{CTC} \rfloor)$ . Performance is measured using Word Error Rate (WER) computed with the `jiwer` toolkit.

**Speech Translation (ST):** Zero-shot speech translation is evaluated on CoVoST2 [37] dataset for English-German (en-de) and English-Chinese (en-zh) split. No translation data is used during training. The LLM output characters are clamped at  $\lfloor 2 \times \text{CTC} \rfloor$ . Performance is measured using BLEU computed with the `SacreBLEU` toolkit<sup>9</sup>.

**Spoken Question Answering (SQA):** We evaluate zero-shot spoken question answering using the IEMOCAP emotion recognition (ER) dataset [38] with the standard 4-class (angry, happy, neutral, and sad) setup using set 5. Performance is reported using classification accuracy. This task is primarily to study the instruction following rate (IFR) of the model, that is, can the model follow new instructions effectively different from what seen during training?

## 5 Results

The closest prior works to DirectSpeech2LLM are LegoSLM, TASU, and AlignFormer. LegoSLM uses CTC posterior probabilities to construct weighted LLM input embeddings and finetunes the LLM on paired speech-text data (ASR and en-de translation). TASU, in contrast, simulates CTC erroneous text to train the adapter using text-only data (ASR and en-zh translation). AlignFormer freezes the LLM and jointly trains the SFM with a dynamic Q-Former adapter using only ASR paired speech text data. Wav2Prompt is related but not directly comparable due to its smaller SFM and an older family of LLM backbone, we only compare its regression loss sensitivity to unseen tasks in Section 5.3.

### 5.1 Main results

Table 1 compares DirectSpeech2LLM against three end-to-end systems on three tasks: ASR, ST, and ER. The only seen task, ASR, during training serves as the fair comparison between end-to-end and cascaded systems.

Despite training on only 1k hours of ASR data, DirectSpeech2LLM outperforms all comparable systems on ASR, and achieves near perfect instruction following on the two unseen tasks ST and ER (IFR = 1.00) with performance closely matching the cascaded system. A cascaded system passes discrete text to the LLM and therefore trivially preserves LLM’s generalization ability. Our method approaches this upper bound zero-shot **corroborating our claim that DirectSpeech2LLM preserves the LLM’s instruction following ability with speech as input, to unseen tasks.** When

<sup>8</sup>[https://github.com/facebookresearch/fairseq/blob/main/examples/mms/data\\_prep/text\\_normalization.py](https://github.com/facebookresearch/fairseq/blob/main/examples/mms/data_prep/text_normalization.py)

<sup>9</sup>SacreBLEU (v2.3.1): `nrefs:1|case:mixed|eff:no|tok:13a,zh|smooth:exp`

Table 1: Models marked with *SFT* uses supervised finetuning for that task. DirectSpeech2LLM shows near perfect instruction following on two unseen tasks, ST and ER as performance approaches to the upper bound of cascade system showing LLM’s generalization ability to new instructions is preserved. IFR, as explained in [13] is shown for SQA task. [(→) means continued training after 38k hours.]

| Model                            | LLM (Learnable) | Train data duration<br>(hrs) | ASR (Librispeech)         | ST (CoVoST2) |             | SQA (IEMOCAP)           |
|----------------------------------|-----------------|------------------------------|---------------------------|--------------|-------------|-------------------------|
|                                  |                 |                              | WER ↓                     | BLEU ↑       |             | ACC ↑                   |
|                                  |                 |                              | t-clean / t-other         | en-de        | en-zh       | Emotion                 |
| Cascaded (CTC + LLM) (50k steps) |                 |                              |                           |              |             |                         |
| Whisper + LLM [11]               | LLama-7B        | -                            | 2.7 / 5.2                 | 18.2         | -           | -                       |
| Ours (Backbone)                  | Phi3.1-3.8B     | 1k                           | 3.13 / 5.12               | 19.37        | 17.52       | 44.76 (IFR = 0.99)      |
|                                  | Qwen3-4B        |                              | 2.35 / 4.57               | 19.88        | 34.03       | 42.98 (IFR = 1.00)      |
| end-to-end (20k steps)           |                 |                              |                           |              |             |                         |
| TASU [27]                        | Qwen2.5-1.5B    | 1k                           | 3.28 / 6.91               | -            | 33.35 (SFT) | -                       |
| LegoSLM [26]                     | Gemma-2B (L)    | 56k                          | - / 5.2                   | 21.1 (SFT)   | -           | -                       |
|                                  |                 | 1k                           | - / 7.0                   | -            | -           | -                       |
| AlignFormer [13]                 | Phi3.1-3.8B     | 38k<br>→ 1k                  | 3.52 / 6.47<br>2.43 / 5.0 | 15.70<br>-   | 10.97<br>-  | 31.18 (IFR = 0.99)<br>- |
| Ours (Backbone)                  | Phi3.1-3.8B     | 1k                           | 2.47 / 4.71               | 16.82        | 16.49       | 37.50 (IFR = 0.99)      |
|                                  | Qwen3-4B        |                              | 2.21 / 4.28               | 19.97        | 33.74       | 40.16 (IFR = 1.00)      |
| Ours (Data size ↑)               | Qwen3-4B        | 2.8k                         | 2.48 / 4.55               | 21.13        | 36.05       | 39.60 (IFR = 1.00)      |
| Ours (Model size ↑)              | Qwen3-1.7B      |                              | 2.4 / 4.43                | 14.12        | 22.87       | 29.52 (IFR = 0.88)      |
|                                  | Qwen3-4B        | 1k                           | 2.21 / 4.28               | 19.97        | 33.74       | 40.16 (IFR = 1.00)      |
|                                  | Qwen3-8B        |                              | 2.07 / 4.06               | 20.12        | 30.35       | 41.45 (IFR = 1.00)      |

evaluated using the identical Phi3.1 backbone as AlignFormer, which enforces temporal but not geometric alignment, our method shows consistent improvement, confirming that the gains stem from the methodology rather than the LLM backbone.

DirectSpeech2LLM also shows consistent scaling behavior: increasing ASR training data from 1k to 2.8k hours further improves zero-shot translation, and scaling the LLM backbone similarly improves performance.

## 5.2 Ablation study

Table 2 ablates the key design choices of DirectSpeech2LLM and their performance on in-domain and out-of-domain seen ASR task and zero-shot unseen ST (en-de) task. We study the effects of commitment loss, scaling the training steps and data, blank token and, LLM backbone.

**Commitment loss:** In stage 1, the model becomes usable only once the commitment loss is used. In stage 2, removing it improves ASR performance and results in marginal degradation on unseen tasks (for results on ER; see Table 6 in Section A.6), suggesting that even the implicit geometric constraint within the CTC loss is sufficient to avoid prompt overfitting. Unlike Wav2Prompt which breaks on unseen tasks if the regression loss is removed as explained in Section 5.3. **This shows that geometric alignment strength plays a smaller role than previously assumed for strong zero-shot performance on unseen tasks.** Nonetheless, we keep using the commitment loss as it consistently provides marginal gains on unseen tasks. Whether it remains necessary in a multitask setup is left for future work.

**Scaling the training steps and data:** Increasing training steps in stage 1 from 20k to 50k and adding more (CV) data across both stages consistently improves both ASR and zero-shot translation, suggesting that more data improves robustness to out-of-domain and unseen tasks. Figure 3 (left) in Section A.5 further shows that ASR performance has not yet saturated in stage 2, indicating room for improvement with additional training steps.

**Role of Blank token:** When the model is trained with no blank token (NB training) there is no prompt overfitting.



Table 2: Ablation study of DirectSpeech2LLM on Stage 1 and Stage 2. Assume Qwen3-4B model and LibriSpeech training data (1k) unless mentioned otherwise. Results compare a cascaded interface (CTC + LLM) with an end-to-end interface. WB and NB corresponds to With blank and No blank setup in training and inference both. Different rows show training setup and columns show inference setup. Hparams (rows) denote the effect of different losses, extended training steps, additional speech data (CV) or different LLM backbone. For stage 2 results, a fair comparison to Linear-CTC loss is Modified-CTC loss  $-\mathcal{L}_{\text{commit}}$ . Gray rows show the best results for each stage. Row marked with (\*) is the closest equivalent to AlignFormer minus the input ordering trick.

| Hparams                                                | ASR (WER ↓)             |                                                                                     |                       |                 | ST (BLEU ↑) |          |
|--------------------------------------------------------|-------------------------|-------------------------------------------------------------------------------------|-----------------------|-----------------|-------------|----------|
|                                                        | Cascaded                | end-to-end                                                                          |                       |                 |             | Cascaded |
|                                                        | CTC → +LLM              | CTC / W <sub>(ith)</sub> B <sub>(lank)</sub> / N <sub>(o)</sub> B <sub>(lank)</sub> |                       | WB / NB         | CTC + LLM   |          |
|                                                        | Libri-other (in-domain) | VP (out-of-domain))                                                                 |                       | CoVoST2 (en-de) |             |          |
| Stage 1: Modified-CTC (20k steps)                      |                         |                                                                                     |                       |                 |             |          |
| → $\mathcal{L}_{\text{ctc}}$                           | 4.88 → 4.86             | - / - / 100.00                                                                      | 16.14 / - / 100.00    | - / 0.00        | 19.89       |          |
| + $\mathcal{L}_{\text{commit}}$                        | 4.94 → 4.85             | - / - / 16.05                                                                       | 16.18 / - / 26.43     | - / 10.44       | 19.92       |          |
| + 30k steps                                            | 4.57 → 4.55             | - / - / 15.91                                                                       | 15.98 / - / 25.24     | - / 13.57       | 19.99       |          |
| → + CV                                                 | 4.81 → 4.80             | - / - / 16.95                                                                       | 14.06 / - / 24.56     | - / 13.67       | 20.98       |          |
| → Phi3.1-3.8B                                          | 4.62 → 10.78            | - / - / 86.48                                                                       | 15.95 / - / 96.23     | - / 0.00        | 19.40       |          |
| Stage 2: Modified-CTC (50k steps) + LLM-CE (20k steps) |                         |                                                                                     |                       |                 |             |          |
| → NB                                                   | 4.63 → 4.62             | - / - / 4.63                                                                        | 15.81 / - / 13.64     | - / 19.02       | 19.92       |          |
| - $\mathcal{L}_{\text{commit}}$                        | 4.62 → 4.61             | - / - / 4.26                                                                        | 15.94 / - / 13.77     | - / 15.87       | 20.00       |          |
| → WB                                                   | 4.60 → 4.57             | - / 4.28 / 5.83                                                                     | 15.95 / 13.86 / 15.09 | 5.80 / 19.97    | 19.88       |          |
| - $\mathcal{L}_{\text{commit}}$                        | 4.50 → 4.49             | - / 4.10 / 5.11                                                                     | 15.92 / 13.71 / 13.61 | 5.59 / 19.69    | 19.96       |          |
| → WB (decoding_E)                                      | - → -                   | - / 7.52 / -                                                                        | - / 15.55 / -         | 19.80 / 19.97   | -           |          |
| → WB + [13]                                            | 4.56 → 4.54             | - / 4.21 / 6.26                                                                     | 15.90 / 13.73 / 15.55 | 16.04 / 19.91   | 19.90       |          |
| → + CV                                                 | 4.88 → 4.85             | - / 4.55 / 7.71                                                                     | 14.18 / 12.56 / 15.01 | 5.01 / 21.13    | 21.02       |          |
| → Phi3.1-3.8B                                          | 4.53 → 5.35             | - / 4.71 / 7.54                                                                     | 15.96 / 14.31 / 15.26 | 1.57 / 16.82    | 19.37       |          |
| Stage 2: Linear-CTC (50k steps) + LLM-CE (20k steps)   |                         |                                                                                     |                       |                 |             |          |
| → NB *                                                 | 4.67 → 4.66             | - / - / 4.3                                                                         | 16.17 / - / 14.29     | - / 1.30        | 20.00       |          |
| → WB                                                   | 4.64 → 4.64             | - / 4.21 / 5.13                                                                     | 16.05 / 13.78 / 14.13 | 1.71 / 18.43    | 19.97       |          |

A model trained with blank token (WB training) shows poor performance on ST task not seen during training, though simply removing the blank embedding at inference fixes this. The blank token is a fixed, shared embedding (“\_”) across all training samples, so the overfitting could be *positional* that is to say the model learns to associate the blank embedding’s fixed position with the ASR task. This hypothesis is confirmed by the *decoding\_E* experiment, moving the blank token to the end during inference restores instruction following. Furthermore when training with more than one task i.e., ASR and en-de combined<sup>10</sup>, positional overfitting of the blank embedding is substantially reduced and performance on en-zh and ER does not collapse, as shown in Table 4 in Section A.3.

Exploring the role of blank embedding to learn utterance level information such as emotion or speaker identity is left for future work.

**LLM backbone:** The stage 1 is sensitive to the LLM backbone used, but performance gap reduces once stage 2 training kicks in. The slight under performance of Phi3.1 compared to Qwen3 after stage 2 training is may be because the model is weaker compared to Qwen3 itself.

**Lastly**, using input ordering trick, from AlignFormer, combined with our method (WB + [13]) reduces prompt overfitting but the performance is poor compared to our method.

**Linear CTC head:** The final two rows in Table 2 evaluate replacing our modified CTC objective with a standard linear CTC head objective. Specifically, we replace the quantization step with a learned linear layer to compute the CTC loss. In the NB training setup, this modification results in

<sup>10</sup>only multitask training setup in this work.

Table 3: The BLEU scores show the sensitivity to the MSE loss in Wav2Prompt or commitment loss in DirectSpeech2LLM. Both are regression loss for geometric alignment.

|                                 | en-es | en-de |
|---------------------------------|-------|-------|
| Wav2Prompt (CIF)                | 13.8  | -     |
| – MSE                           | 5.4   | -     |
| DirectSpeech2LLM (Modified CTC) | -     | 19.97 |
| – $\mathcal{L}_{\text{commit}}$ | -     | 19.69 |

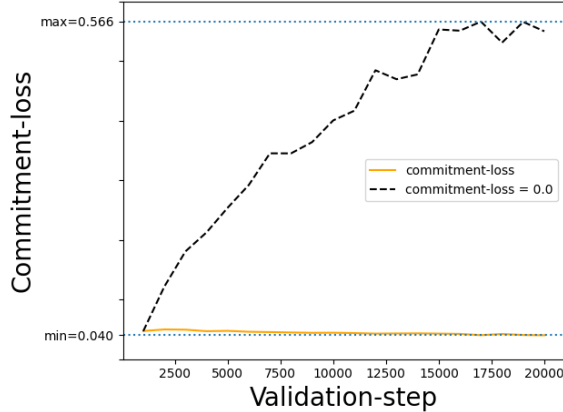


Figure 2: Validation commitment loss of DirectSpeech2LLM with and without commitment loss.

substantial degradation on the unseen ST task, consistent with the prompt overfitting observed in prior work. On the contrary in the WB training setup, the degradation is largely mitigated by removing the blank embedding during inference (NB decoding).

These observations suggest that the free-floating SFM-OES is learning task-specific cues within those embeddings, a phenomenon commonly referred to as *shortcut learning* [39]. In the case of WB training setup, these task-specific cues are shown to be majorly restricted to the blank token only.

### 5.3 Regression loss sensitivity of Wav2Prompt vs DirectSpeech2LLM on unseen task

As shown in Table 3, Wav2Prompt [7] exhibits extreme sensitivity to its geometric alignment objective on the performance on unseen tasks, removing the MSE loss causes catastrophic degradation in zero-shot translation performance (BLEU drops from 13.8 to 5.4 for en-es language). In contrast, DirectSpeech2LLM degrades only marginally when the commitment loss is removed (19.97  $\rightarrow$  19.69 for en-de language).

Figure 2 shows this geometric divergence, despite a  $10\times$  increase in embedding distances, zero-shot performance remains stable. One possible explanation is that positive-pair alignment alone does not guarantee sufficient separation between token embeddings. The LLM requires the target embeddings to remain sufficiently distinct to reliably discriminate between tokens. Thus, minimizing the distance to the positive target can be insufficient if it simultaneously reduces separation from competing embeddings. We leave a deeper investigation of this phenomenon to future work.

Since geometric alignment is already implicitly enforced in our modified CTC objective through logits computed as negative squared distances, DirectSpeech2LLM removes the need for an additional hyperparameter. Consequently, it provides a more principled mechanism that is less sensitive to hyperparameter tuning while also eliminating the train, inference mismatch introduced by CIF’s length dependent weight scaling.

## 6 Conclusion and Future work

We introduce DirectSpeech2LLM, a simple end-to-end framework to align the SFM-OES to the LLM-IES of a pretrained LLM along two dimensions: temporal and geometric. This is achieved using a modified CTC objective with vector quantization. It achieves strong ASR performance and demonstrates robust instruction following on two unseen tasks, ST and ER, despite being trained solely on English ASR data. These results suggest that our method provides a simple and principled framework for integrating speech to LLM while preserving its generalization ability to unseen tasks.

**Limitations and Future work:** Because DirectSpeech2LLM is trained on a single ASR task, its behavior in multilingual and multi-task settings remains underexplored. The role of blank and non blank embedding(s), particularly when jointly optimized with paralinguistic tasks, also remains unclear. Additionally, temporal alignment uses CTC greedy decoding, which may limit performance relative to more expressive decoding strategies, such as beam search augmented with an n-gram language model. The SFM, Data2Vec, is pretrained using a student-teacher setup, though we use only the student part in this work, in future we will extend the method to use teacher’s pseudo labels to scale the training data in cases where we do not have the target for computing CTC loss, a semi-supervised setup. This includes cases where we have source speech and LLM targets but not source transcriptions.

## References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Thomas Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeff Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Ma teusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *ArXiv*, abs/2005.14165, 2020. URL <https://api.semanticscholar.org/CorpusID:218971783>.
- [2] Hugo Touvron, Thibaut Lavril, Girish Sastry, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [3] OpenAI. Gpt-4 technical report. 2023. URL <https://api.semanticscholar.org/CorpusID:257532815>.
- [4] Jing Peng, Yucheng Wang, Bohan Li, Yiwei Guo, Hankun Wang, YanGui Fang, Yu Xi, Haoyu Li, Xu Li, Ke Zhang, Shuai Wang, and Kai Yu. A survey on speech large language models for understanding. *IEEE Journal of Selected Topics in Signal Processing*, 20(1):2–31, January 2026. ISSN 1941-0484. doi: 10.1109/jstsp.2025.3640535. URL <http://dx.doi.org/10.1109/JSTSP.2025.3640535>.
- [5] Hemant Yadav, Rajiv Ratn Shah, and Sunayana Sitaram. Jooci: a framework for learning comprehensive speech representations. *arXiv preprint arXiv:2410.11086*, 2024.
- [6] Jingyi Chen, Zhimeng Guo, Jiyun Chun, Pichao Wang, Andrew Perrault, and Micha Elsner. Do audio llms really listen, or just transcribe? measuring lexical vs. acoustic emotion cues reliance, 2025. URL <https://arxiv.org/abs/2510.10444>.
- [7] Keqi Deng, Guangzhi Sun, and Phil Woodland. Wav2prompt: End-to-end speech prompt generation and tuning for llm in zero and few-shot learning. *ArXiv*, abs/2406.00522, 2024. URL <https://api.semanticscholar.org/CorpusID:270220534>.
- [8] Heting Gao, Junrui Ni, Kaizhi Qian, Yang Zhang, Shiyu Chang, and Mark Hasegawa-Johnson. Wavprompt: Towards few-shot spoken language understanding with frozen language models, 2022. URL <https://arxiv.org/abs/2203.15863>.
- [9] Jian Wu, Yashesh Gaur, Zhuo Chen, Long Zhou, Yimeng Zhu, Tianrui Wang, Jinyu Li, Shujie Liu, Bo Ren, Linqun Liu, et al. On decoder-only architecture for speech-to-text and large language model integration. In *2023 IEEE automatic speech recognition and understanding workshop (ASRU)*, pages 1–8. IEEE, 2023.

- [10] Ziyang Ma, Guanrou Yang, Yifan Yang, Zhifu Gao, Jiaming Wang, Zhihao Du, Fan Yu, Qian Chen, Siqi Zheng, Shiliang Zhang, and Xie Chen. An embarrassingly simple approach for llm with strong asr capacity, 2024. URL <https://arxiv.org/abs/2402.08846>.
- [11] Shujie Hu, Long Zhou, Shujie Liu, Sanyuan Chen, Lingwei Meng, Hongkun Hao, Jing Pan, Xunying Liu, Jinyu Li, Sunit Sivasankaran, et al. Wavllm: Towards robust and adaptive speech large language model. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 4552–4572, 2024.
- [12] Changli Tang, Wenyi Yu, Guangzhi Sun, Xianzhao Chen, Tian Tan, Wei Li, Lu Lu, Zejun Ma, and Chao Zhang. Salmonn: Towards generic hearing abilities for large language models. *arXiv preprint arXiv:2310.13289*, 2023.
- [13] Ruchao Fan, Bo Ren, Yuxuan Hu, Rui Zhao, Shujie Liu, and Jinyu Li. Alignformer: Modality matching can achieve better zero-shot instruction-following speech-llm. *arXiv preprint arXiv:2412.01145*, 2024.
- [14] Santiago Cuervo, Skyler Seto, Maureen de Seyssel, Richard He Bai, Zijin Gu, Tatiana Likhomanenko, Navdeep Jaitly, and Zakaria Aldeneh. Closing the gap between text and speech understanding in llms, 2026. URL <https://arxiv.org/abs/2510.13632>.
- [15] Biswesh Mohapatra, Marcely Zanon Boito, and Ioan Calapodescu. Speechmapper: Speech-to-text embedding projector for llms. *arXiv preprint arXiv:2601.20417*, 2026.
- [16] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*, pages 28492–28518. PMLR, 2023.
- [17] Dong Zhang, Shimin Li, Xin Zhang, Jun Zhan, Pengyu Wang, Yaqian Zhou, and Xipeng Qiu. Speechgpt: Empowering large language models with intrinsic cross-modal conversational abilities. *arXiv preprint arXiv:2305.11000*, 2023.
- [18] Jing Pan, Jian Wu, Yashesh Gaur, Sunit Sivasankaran, Zhuo Chen, Shujie Liu, and Jinyu Li. Cosmic: Data efficient instruction-tuning for speech in-context learning. *arXiv preprint arXiv:2311.02248*, 2023.
- [19] Yunfei Chu, Jin Xu, Xiaohuan Zhou, Qian Yang, Shiliang Zhang, Zhijie Yan, Chang Zhou, and Jingren Zhou. Qwen-audio: Advancing universal audio understanding via unified large-scale audio-language models, 2023. URL <https://arxiv.org/abs/2311.07919>.
- [20] Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanus Phillips, Irena Gao, et al. Wilds: A benchmark of in-the-wild distribution shifts. In *International conference on machine learning*, pages 5637–5664. PMLR, 2021.
- [21] Shashi Kumar, Iuliia Thorbecke, Sergio Burdisso, Esaú Villatoro-Tello, Manjunath KE, Kadri Hacıoğlu, Pradeep Rangappa, Petr Motlicek, Aravind Ganapathiraju, and Andreas Stolcke. Performance evaluation of slam-asr: The good, the bad, the ugly, and the way forward. In *2025 IEEE International Conference on Acoustics, Speech, and Signal Processing Workshops (ICASSPW)*, pages 1–5. IEEE, 2025.
- [22] Junseok Oh and Ji-Hwan Kim. Seam: Bridging the temporal-semantic granularity gap for llm-based speech recognition. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 2135–2144, 2026.
- [23] Chen Wang, Minpeng Liao, Zhongqiang Huang, Jinliang Lu, Junhong Wu, Yuchen Liu, Chengqing Zong, and Jiajun Zhang. Blsp: Bootstrapping language-speech pre-training via behavior alignment of continuation writing. *arXiv preprint arXiv:2309.00916*, 2023.
- [24] Aohan Zeng, Zhengxiao Du, Mingdao Liu, Lei Zhang, Shengmin Jiang, Yuxiao Dong, and Jie Tang. Scaling speech-text pre-training with synthetic interleaved data, 2024. URL <https://arxiv.org/abs/2411.17607>.

- [25] Yuhao Zhang, Zhiheng Liu, Fan Bu, Ruiyu Zhang, Benyou Wang, and Haizhou Li. Soundwave: Less is more for speech-text alignment in llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18718–18738, 2025.
- [26] Rao Ma, Tongzhou Chen, Kartik Audhkhasi, and Bhuvana Ramabhadran. Legoslm: Connecting llm with speech encoder using ctc posteriors. *arXiv preprint arXiv:2505.11352*, 2025.
- [27] Jing Peng, Yi Yang, Xu Li, Yu Xi, Quanwei Tang, Yangui Fang, Junjie Li, and Kai Yu. Tasu: Text-only alignment for speech understanding. *arXiv preprint arXiv:2511.03310*, 2025.
- [28] Ioannis Tsiamas, Gerard I Gállego, José AR Fonollosa, and Marta R Costa-jussà. Pushing the limits of zero-shot end-to-end speech translation. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 14245–14267, 2024.
- [29] Phuong-Hang Le, Hongyu Gong, Changhan Wang, Juan Pino, Benjamin Lecouteux, and Didier Schwab. Pre-training for speech translation: Ctc meets optimal transport. In *International Conference on Machine Learning*, pages 18667–18685. PMLR, 2023.
- [30] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. *Advances in Neural Information Processing Systems*, 30, 2017.
- [31] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation, 2021. URL <https://arxiv.org/abs/2101.00190>.
- [32] Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin Dogus Cubuk, and Quoc V. Le. Specaugment: A simple data augmentation method for automatic speech recognition. *ArXiv*, abs/1904.08779, 2019. URL <https://api.semanticscholar.org/CorpusID:121321299>.
- [33] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur. Librispeech: an asr corpus based on public domain audio books. In *Acoustics, Speech and Signal Processing (ICASSP), 2015 IEEE International Conference on*, pages 5206–5210. IEEE, 2015.
- [34] Rosana Ardila, Megan Branson, Kelly Davis, Michael Kohler, Josh Meyer, Michael Henretty, Reuben Morais, Lindsay Saunders, Francis Tyers, and Gregor Weber. Common voice: A massively-multilingual speech corpus. In *Proceedings of the twelfth language resources and evaluation conference*, pages 4218–4222, 2020.
- [35] Alexei Baevski, Wei-Ning Hsu, Qiantong Xu, Arun Babu, Jiatao Gu, and Michael Auli. Data2vec: A general framework for self-supervised learning in speech, vision and language. In *International conference on machine learning*, pages 1298–1312. PMLR, 2022.
- [36] Changhan Wang, Morgane Riviere, Ann Lee, Anne Wu, Chaitanya Talnikar, Daniel Haziza, Mary Williamson, Juan Pino, and Emmanuel Dupoux. VoxPopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 993–1003, Online, August 2021. Association for Computational Linguistics. URL <https://aclanthology.org/2021.acl-long.80>.
- [37] Changhan Wang, Anne Wu, and Juan Pino. Covost 2: A massively multilingual speech-to-text translation corpus, 2020.
- [38] Carlos Busso, Murtaza Bulut, Chi-Chun Lee, Abe Kazemzadeh, Emily Mower, Samuel Kim, Jeannette N Chang, Sungbok Lee, and Shrikanth S Narayanan. Iemocap: Interactive emotional dyadic motion capture database. *Language resources and evaluation*, 42(4):335–359, 2008.
- [39] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.

- [40] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 369–376, 2006.
- [41] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM transactions on audio, speech, and language processing*, 29:3451–3460, 2021.
- [42] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. Soundstream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2021.
- [43] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9630–9640, 2021. URL <https://api.semanticscholar.org/CorpusID:233444273>.
- [44] Alexander H Liu, Heng-Jui Chang, Michael Auli, Wei-Ning Hsu, and Jim Glass. Dinosr: Self-distillation and online clustering for self-supervised speech representation learning. *Advances in Neural Information Processing Systems*, 36:58346–58362, 2023.
- [45] Hao Liu, Wilson Yan, and Pieter Abbeel. Language quantized autoencoders: Towards unsupervised text-image alignment. *Advances in neural information processing systems*, 36:4382–4395, 2023.
- [46] Fabian Mentzer, David Minnen, Eirikur Agustsson, and Michael Tschannen. Finite scalar quantization: Vq-vae made simple. *arXiv preprint arXiv:2309.15505*, 2023.
- [47] Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, et al. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258, 2025.
- [48] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. Byt5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.
- [49] Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. Megabyte: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems*, volume 36, pages 78808–78823, 2023.

## A Appendix

### A.1 Preliminaries

#### A.1.1 Large Language Model (LLM) Input

LLMs process language by mapping raw text into sequences of discrete tokens using a fixed set of rules (temporal alignment). Each token is then mapped to a high-dimensional embedding (geometric alignment). Together, these define two dimensions along which the input can deviate from the seen input distribution: the token sequence and the embedding space. While LLMs tolerate small perturbations, changes to either can significantly degrade performance [15, 20], resulting in a loss of its generalization to follow instructions.

Therefore, constructing an effective Speech-LLM requires satisfying two alignment criteria: temporal and geometric. **Our hypothesis** is that minimizing distribution along these two dimensions **may** improve cross-modal alignment resulting in increased generalization to unseen tasks.

#### A.1.2 Connectionist Temporal Classification (CTC)

CTC [40] is a sequence learning objective designed for tasks where the exact alignment between input frames and output tokens is unknown, such as ASR. Given an input sequence of acoustic representations  $\mathbf{X}$  and a target token sequence  $\mathbf{Y}$ , CTC models the conditional probability by marginalizing over all possible frame level alignments that collapse to  $\mathbf{Y}$ . To enable flexible alignment, CTC introduces a special *blank* symbol that allows the model to emit no valid token at a given timestep. During inference, repeated tokens and blanks are removed through a collapsing operation to produce the final transcription.

#### A.1.3 Vector Quantization (VQ)

VQ [30] is a process of committing continuous representations to a discrete codebook, which in this work is identified with the LLM input embedding space (IES). Given an SFM output  $\mathbf{z}$ , the quantizer selects the nearest codebook vector  $\mathbf{q}_k \in \mathcal{E} = \mathbf{q}_1, \dots, \mathbf{q}_K$ , encouraging the representation to align with its quantized counterpart before passing it downstream. The use of vector quantization in speech [41, 42] or in vision [43] representation learning is well established. However, existing methods primarily treat the codebook as an auxiliary component for generating pseudo-labels, or learn it jointly during training [44].

Unlike standard VQ, which learns a dynamic codebook, we instead fix the codebook to be LLM-IES, following approaches such as [45, 46]. We reinterpret quantization as a classification problem, where mean squared error scores are used as logits, reducing the quantizer to a direct prediction layer over codebook indices, providing a stable interface for end-to-end integration between speech and the language model.

### A.2 Training Procedure

**Stage 1:** The SFM is trained using the modified CTC objective and the commitment loss as described in Section 3. During this stage, the last six layers of the `Data2Vec-Large` are finetuned. Training runs for 50k steps and the peak learning rate (LR) is  $1 \times 10^{-4}$ . The LR is linearly warm-up for the first 10% of the steps and then linearly decayed for the remaining steps.

**Stage 2:** We add cross-entropy loss from LLM to stage 1 losses, while keeping the LLM frozen. At this stage, only the last two layers of the `Data2Vec-Large` encoder are finetuned. Training runs for 20k steps and the peak LR is  $5 \times 10^{-5}$ . The LR is linearly warm-up for the first 10% of the steps and kept constant for 80% of the steps and then linearly decay for the remaining steps.

Both stages use the AdamW optimizer with gradient clipping set to 1 using 4 B200 GPUs with 180 GB memory. Each GPU processes at max 200 seconds of audio per step and the total batch has at max 800 seconds. All experiments are conducted using *TF32* precision. Stage 1 (50k steps) completes in approximately 2.5 hours and Stage 2 (20k steps) in approximately 2 hours for the Qwen3-4B configuration.

### A.3 Multitask Training Setup

Table 4 reports results under a multitask training configuration in which the model is jointly trained on LibriSpeech ASR data and English-to-German speech translation (en-de) data from CoVoST2. Commitment loss is excluded in this setup to isolate the effect of multitask supervision.

The primary finding is that positional overfitting to the blank token embedding, a limitation of the single task WB setup is substantially reduced when more than one task is present during training. The model learns to treat the blank token as a general purpose utterance level embedding rather than a fixed cue for the ASR task. Notably, the WB setup in the multitask configuration outperforms the cascaded system on all seen tasks. This indicates that end-to-end training with a contextual blank embedding is advantageous over the cascaded pipeline.

Table 4: Multitask training setup with Librispeech and en-de training data without commitment loss.

| Tasks       | WB    | WB <sub>decoding_E</sub> | NB    | CTC + LLM |
|-------------|-------|--------------------------|-------|-----------|
| Libri-clean | 2.4   | 2.95                     | 2.8   | 2.39      |
| Libri-other | 4.39  | 5.73                     | 5.51  | 4.61      |
| en-de       | 21.45 | 21.61                    | 21.68 | 20.29     |
| en-zh       | 34.20 | 33.91                    | 35.23 | 34.98     |
| Emotion     | 40.89 | 39.35                    | 41.53 | 43.55     |

### A.4 Longform Speech Inference

We evaluate DirectSpeech2LLM on longform speech by concatenating LibriSpeech test-clean utterances of duration ranging from 1 to 20 minutes. The model is trained on utterances of upto 20 seconds; evaluation on longer sequences thus constitutes out-of-distribution inference with respect to sequence length.

The results in Table 5 reveal a clear degradation pattern as duration increases. At 1 minute, both WB and NB setups match the CTC baseline. By 10 minutes, WB degrades substantially (WER 14.26), while NB remains comparatively robust (WER 4.22). At 20 minutes, all setups degrade significantly.

This behavior reflects a well known limitation of transformer self-attention, attention patterns do not extrapolate reliably to sequence lengths beyond the training distribution. The blank embedding in the WB setup compounds this issue by encoding a fixed positional cue that becomes increasingly misaligned to longer durations. Restricting the SFM to a fixed sliding-window attention mechanism (e.g., 20-60 second windows) is a natural remedy that we leave this to future work.

Table 5: Longform speech inference results.

| Duration | WB    | NB    | CTC   |
|----------|-------|-------|-------|
| 1 min    | .62   | 0.62  | 0.62  |
| 5 min    | 2.97  | 2.6   | 3.71  |
| 10 min   | 14.26 | 4.22  | 5.08  |
| 20 min   | 38.48 | 33.12 | 12.37 |

### A.5 Training dynamics

Figure 3 presents two views of training dynamics across the Qwen3-4B and Phi-3.1-3.8B LLM backbones.

#### A.5.1 WER Convergence (Figure 3, Left)

The left panel shows Stage 2 LLM WER on LibriSpeech test-other as a function of training step. Both backbones exhibit sustained improvement throughout the 20,000 step training budget, with no sign of convergence plateauing. These trends indicate that performance gains from extended training are likely available for both backbones, and that our reported results represent a lower bound on what is achievable with the current framework.



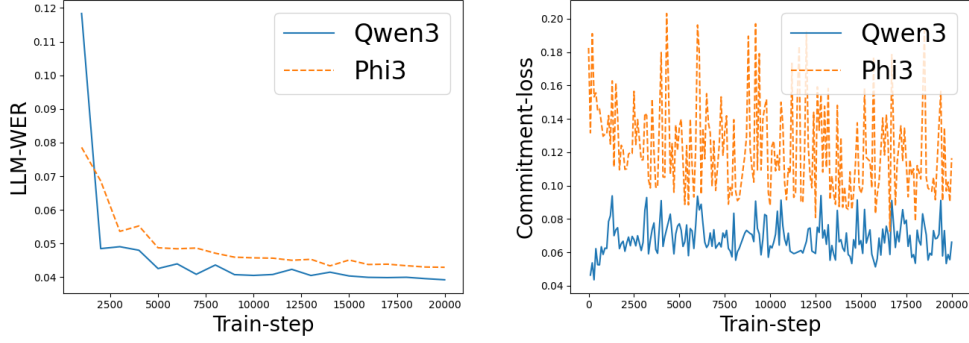


Figure 3: Left: Stage 2 LLM WER vs LLM backbone shows training longer would improve performance further. Right: Commitment loss vs LLM backbone.

#### A.5.2 Commitment Loss (Figure 3, Right)

The right panel shows the commitment loss trajectory during Stage 2 training. Phi-3.1-3.8B exhibits substantially higher and more erratic commitment loss values compared to Qwen3-4B, which converges to a consistently lower loss with less variance. This indicates that the SFM outputs align more easily with the Qwen3-4B embeddings geometry. The underlying cause of this difference whether it stems from the size, training data, or architectural characteristics of each LLM remains an open question that we leave for future investigation.

Taken together, these training dynamics reinforce two conclusions from the main paper: (i) Direct-Speech2LLM’s performance has not saturated and stands to benefit from longer training, and (ii) LLM backbone quality is a meaningful factor in the geometric alignment between the SFM output space and the LLM input embedding space.

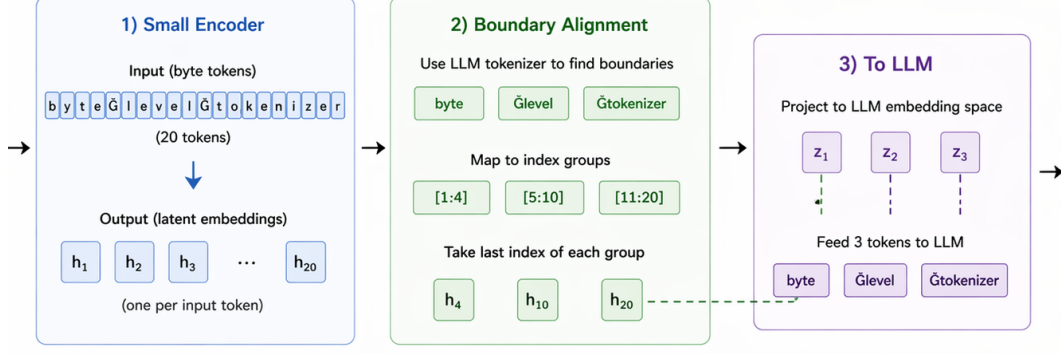


Figure 4: TBP method overview. The authors of Byte Latent Transformer [47] use dynamic boundaries at step 2, while TBP uses static tokenizer dependent boundaries.

### A.6 Token Boundary Preservation

We propose Token Boundary Preservation (TBP), implemented via a Bytes-to-Token Causal encoder (BTC) that aggregates byte-level representations according to tokenizer-specific token spans.

Consider the input text "byte level tokenizer". Qwen3 LLM tokenizer splits this string into 3 tokens:

$$[\text{byte}, \text{level}, \text{tokenizer}].$$

In contrast, the Speech-LLM tokenizer of DirectSpeech2LLM method operating on a restricted vocabulary of 1K splits this string into 9 tokens:

$$[\text{by}, \text{te}, \text{le}, \text{vel}, \text{to}, \text{k}, \text{en}, \text{iz}, \text{er}].$$

Similarly, a pure byte-level tokenizer splits this string into 20 tokens:

$$[b, y, t, e, \text{level}, l, e, v, e, l, \text{tokenizer}, t, o, k, e, n, i, z, e, r].$$

Our objective is to preserve the tokenization spans seen by the LLM during pre-training (i.e., the original 3 tokens) rather than feeding different sequences of 9 or 20 tokens. TBP uses the LLM’s tokenizer to define deterministic *token spans* over byte-level representations, different from BLT’s stochastic spans which introduces additional complexity. Formally, let the BTC encoder process the byte-level sequence:

$$[b, y, t, e, \text{level}, l, e, v, e, l, \text{tokenizer}, t, o, k, e, n, i, z, e, r]$$

to produce latent embeddings  $\{h_1, \dots, h_{20}\}$ . Applying the LLM tokenizer to the original text yields token spans corresponding to:

$$[\text{byte}, \text{level}, \text{tokenizer}],$$

which map to byte-index ranges  $[1:4]$ ,  $[5:10]$ , and  $[11:20]$ . Using these spans, the BTC encoder aggregates representations by selecting the final latent vector from each span:

$$\{h_4, h_{10}, h_{20}\}.$$

These representations are then projected into the LLM input embedding space and passed as input to the LLM. To ensure compatibility with the LLM input embedding space (IES), BTC encoder logits are calculated similar to the Stage 1 of DirectSpeech2LLM but instead of CTC we use cross-entropy loss. An overview of the TBP is illustrated in Figure 4 and the updated DirectSpeech2LLM\_TBP method is illustrated in Figure 5. The results are shown in Table 6 and 7.

Because BTC embeddings are conditioned on raw bytes, they might be more robust to spelling errors and better at tasks which require character-level granularity, such as counting the "r"s in "strawberry" or solving math. Furthermore, we plan to explore using multiple TBP rules with prefix tuning, for a single BTC. In multilingual settings, this would allow us to dynamically choose the TBP rule that produces the fewest tokens for a given language.

Table 6: Extension to Table 1. Numbers with underline shows the best result for each test set. The improvements except on the ASR task are marginal. Overall, the results suggest that input tokenization has no substantial effect on the LLM’s task-solving ability. Values in parentheses report ASR results computed using the Whisper EnglishTextNormalizer compared to mms normalization.

| -                                   | -         | -  | t-clean / t-other                       | en-de        | en-zh        | Emotion                   |
|-------------------------------------|-----------|----|-----------------------------------------|--------------|--------------|---------------------------|
| <b>end-to-end</b>                   |           |    |                                         |              |              |                           |
| Ours                                |           |    | 2.21 (2.05) / 4.28 (4.13)               | <u>19.97</u> | <u>33.74</u> | 40.16 (IFR = 1.00)        |
| Ours $-\mathcal{L}_{\text{commit}}$ | Qwen-3-4B | 1k | 2.13 (1.96) / 4.09 (3.93)               | 19.69        | 32.86        | 39.52 (IFR = 1.00)        |
| Ours + TBP                          |           |    | <u>1.96</u> (1.81) / <u>4.06</u> (3.90) | 19.79        | 32.74        | <u>40.97</u> (IFR = 1.00) |

Table 7: Extension to Table 2. Numbers with underline shows the best result for each test set.

| Hparams  | ASR (WER ↓)             |                                                   |                       | ST (BLEU ↑)                        |                            |
|----------|-------------------------|---------------------------------------------------|-----------------------|------------------------------------|----------------------------|
|          | Cascaded                | end-to-end                                        |                       | Cascaded                           |                            |
|          | CTC → +LLM              | CTC / $W_{(t h)}B_{(lank)}$ / $N_{(o)}B_{(lank)}$ |                       | WB / NB                            | CTC + LLM                  |
|          | Libri-other (in-domain) | VP (out-of-domain)                                |                       | CoVoST2 (en-de)                    |                            |
| Stage 2  |                         |                                                   |                       |                                    |                            |
| Ours     | 4.60 → 4.57             | - / 4.28 / 5.83                                   | 15.95 / 13.86 / 15.09 | 5.80 / 19.97 (49.4/26.1/15.1/9.1)  | 19.88 (48.8/25.7/14.8/8.9) |
| Ours_TBP | 4.38 → 4.38             | - / 4.06 / 4.43                                   | 15.74 / 13.89 / 14.42 | 11.44 / 19.79 (49.0/25.8/15.0/9.0) | 20.10 (49.3/26.2/15.2/9.1) |

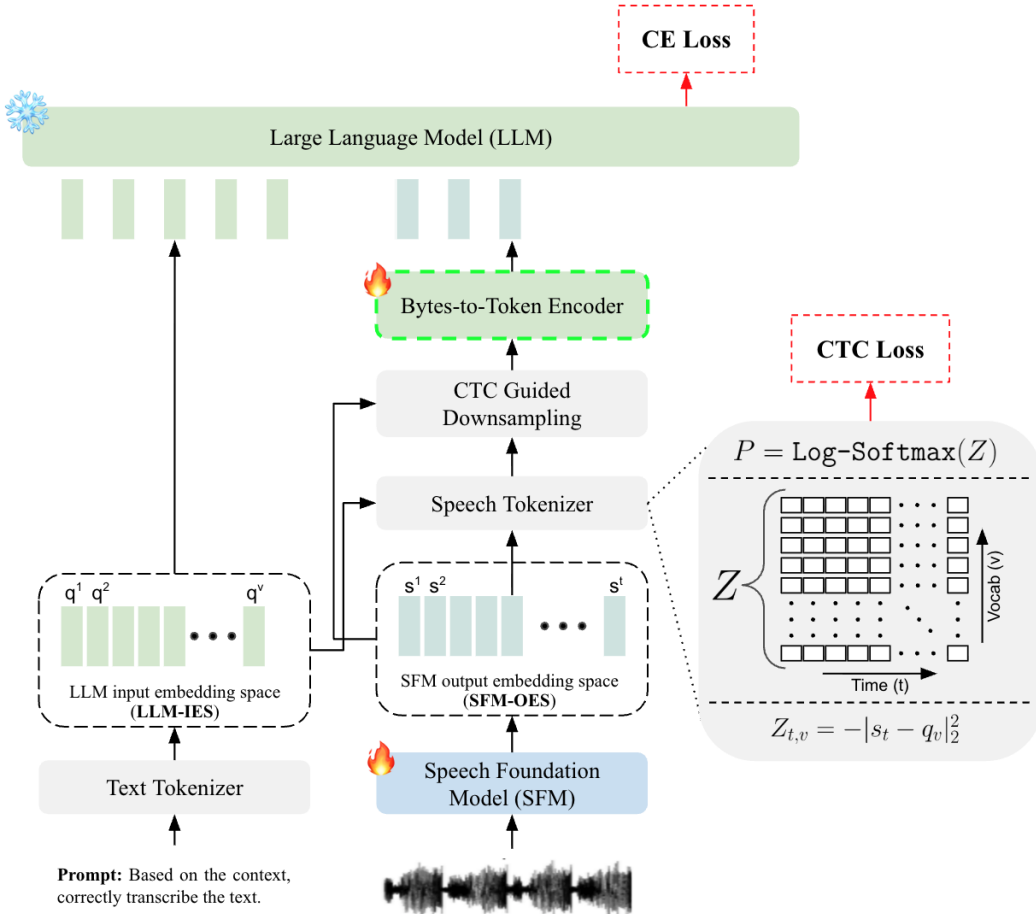


Figure 5: Updated DirectSpeech2LLM\_TBP method.

## A.7 Pseudocode

Algorithm 1 provides the full training procedure for DirectSpeech2LLM. The key steps are: (i) computing distance-based logits over the frozen LLM embedding matrix, (ii) deriving greedy CTC labels for guided downsampling and commitment loss, and (iii) optionally applying LLM cross-entropy in Stage 2.

Algorithm 2 describes the construction of the speech tokenizer vocabulary  $\mathcal{V}_{\text{ctc}}$  as the intersection of a BPE vocabulary trained on ASR transcripts and the LLM vocabulary.

Note that the commitment loss is clamped to never exceed the CTC loss. Because commitment targets are derived from greedy CTC predictions, a poorly calibrated CTC head can produce noisy labels, which would push SFM embeddings toward incorrect geometric anchors. By clamping the commitment loss, we ensure that it cannot dominate training before CTC has converged sufficiently.

---

### Algorithm 1 DirectSpeech2LLM method

---

**Require:** Training set  $\mathcal{D} = \{(X^{(i)}, Y^{(i)})\}_{i=1}^N$

**Require:** SFM  $E_\phi$  with output dimension  $d_{\text{llm}}$

**Require:** LLM  $\mathcal{M}_\theta$  and its embedding matrix  $Q \in \mathbb{R}^{|\mathcal{V}_{\text{llm}}| \times d_{\text{llm}}}$

**Require:** Speech tokenizer vocab  $\mathcal{V}_{\text{ctc}} \subset \mathcal{V}_{\text{llm}}$

**Require:** Blank index  $b$

**Require:** Loss weights  $\lambda_{\text{llm}}$

```

1: for each training iteration do
2:    $S \leftarrow E_\phi(X)$   $\triangleright S \in \mathbb{R}^{B \times T \times d_{\text{llm}}}$ 
3:   Compute logits,  $Z$ , using

$$Z_{t,v} = -(\|s_t\|_2^2 - 2s_t^\top q_v + \|q_v\|_2^2)$$

4:    $P \leftarrow \text{Log-Softmax}(Z)$   $\triangleright P \in \mathbb{R}^{B \times T \times \mathcal{V}_{\text{llm}}}$ 
5:    $P_{\text{ctc}} \leftarrow P[:, :, \mathcal{V}_{\text{ctc}}]$   $\triangleright$  Gather valid  $\mathcal{V}_{\text{ctc}}$  log-prob
6:    $\mathcal{L}_{\text{ctc}} \leftarrow \text{CTC\_Loss}(P_{\text{ctc}}, Y)$ 

7:    $\hat{I} \leftarrow \arg \max_{v \in \mathcal{V}_{\text{ctc}}} P_{\text{ctc}}$   $\triangleright$  Greedy token indices
8:    $S_q \leftarrow Q[\hat{I}]$   $\triangleright$  Quantized embeddings
9:    $M \leftarrow \mathbb{1}[\hat{I}_t \neq b]$   $\triangleright$  Non blank mask
10:   $\mathcal{L}_{\text{commit}} \leftarrow \frac{\|(S - S_q) \odot M\|_F^2}{\sqrt{d_{\text{llm}}}}$ 
11:   $S_{\text{seg}} \leftarrow \text{CTC-Guided-Downsampling}(S, \hat{I})$ 

12:   $E_p \leftarrow Q(\text{Prompt})$   $\triangleright$  Prompt embeddings
13:   $\mathcal{L}_{\text{llm}} \leftarrow \text{CrossEntropy}(\mathcal{M}_\theta(E_p, S_{\text{seg}}), Y)$ 

14:   $\mathcal{L} \leftarrow \mathcal{L}_{\text{ctc}} + \mathcal{L}_{\text{commit}} + (\mathcal{L}_{\text{llm}} \times \lambda_{\text{llm}})$ 
15:  Update  $\phi$  using  $\nabla \mathcal{L}$ 
16: end for

```

---

### Algorithm 2 Speech tokenizer vocabulary

---

**Require:** Text corpus  $\mathcal{C}$ , target vocab size  $K$

**Require:** LLM tokenizer with vocabulary  $\mathcal{V}_{\text{llm}}$

- 1: Train a BPE tokenizer on  $\mathcal{C}$  with size  $K$
  - 2: Obtain vocabulary  $\mathcal{V}_{\text{bpe}}$  and merges  $\mathcal{M}_{\text{bpe}}$
  - 3:  $\mathcal{V}_{\text{ctc}} \leftarrow \mathcal{V}_{\text{bpe}} \cap \mathcal{V}_{\text{llm}}$
  - 4:  $\mathcal{M}_{\text{ctc}} \leftarrow \{(a, b) \in \mathcal{M}_{\text{bpe}} \mid a, b, (a||b) \in \mathcal{V}_{\text{ctc}}\}$
  - 5: Construct speech tokenizer using  $\mathcal{V}_{\text{ctc}}$  and  $\mathcal{M}_{\text{ctc}}$
  - 6: **return** speech tokenizer defining  $\mathcal{V}_{\text{ctc}} \subset \mathcal{V}_{\text{llm}}$
-

## A.8 Task prompts

The template used for Qwen-3 LLM is shown in Figure 6.

```

Prompt Template

<|im_start|>system
You are a helpful Multilingual conversational assistant.
<|im_end|>
<|im_start|>user
TASK SPECIFIC PROMPT
<context>
SFM OUTPUT EMBEDDINGS
</context>
<|im_end|>
<|im_start|>assistant
<think>\n
</think>\n
...LLM RESPONSE ...

Task Specific Prompts

ASR: Based on the context, correctly repeat the text without any explanations
or formatting.
ST: Based on the context, correctly translate the text into German/Chinese.
Output only the German/Chinese translation without any explanations or
formatting.
ER: Based on the context, identify the emotion from the following categories:
angry, happy, neutral, and sad. Output only the emotion category without any
explanations or formatting.
```

Figure 6: Prompt template used for Stage 2 training and inference with Qwen3, including task-specific prompts for ASR, speech translation, and emotion recognition.

## A.9 Discussion

Failure examples from Stage 1 using no-blank (NB) decoding for the ASR task are shown in Table 8. These failures are due to the input distribution mismatch between the CTC and LLM tokenizers or sometimes the LLM ignores the instruction and starts translating. These errors are eliminated when using the same decoding strategy as used during training, WB or NB.

One potential direction to address the formatting errors is to use byte-level LLMs [48, 49]. Although byte-level modeling increases sequence length, recent work such as the Byte Latent Transformer (BLT) [47] addresses this by learning latent token representations that compress byte sequences dynamically while preserving modeling capacity providing a promising direction for bridging CTC style character level embeddings with language models. Similar to [28], we propose a simple solution in Section A.6, called TBP, to make any LLM take raw-bytes/characters as input and therefore extending it to DirectSpeech2LLM. The results are shown in Table 6 with marginal improvement on the ASR task and no clear gains on the unseen tasks.

Our method’s sensitivity to the duration of input speech seen in training vs inference is shown in A.4. During training it sees speech upto 20 seconds and is able to do inference upto 300 seconds after which the performance lags behind cascaded system.

Lastly, it is easier to align spoken text temporally with a LLM. However, it is non-trivial to align audio with a LLM which do not have any direct temporal relation to align, especially if the LLM was never exposed to audio during pre-training. Expecting such a model to suddenly perform multimodal instruction following is unrealistic. It is analogous to bypassing the foundational text pre-training phase in LLM development, without learning the basic structure and representations of the modality, consistent performance is not achievable.

Table 8: Dominant failure cases of DirectSpeech2LLM on the ASR task when using the NB decoding strategy. Stage 1 formatting failures are expected as we ask the LLM to repeat. Even after stage 2 LLM finetuning, the formatting failures of limited vocabulary of speech tokenizer are still visible, very rare. We also found degradation in instruction following as sometimes, instead of repeating, the llm starts translating.

|                                                                                                                              |
|------------------------------------------------------------------------------------------------------------------------------|
| <b>Stage 1 – Failure: Formatting</b>                                                                                         |
| <b>Real:</b> i’d recommend him to you instead of blackstone thanks laughed kenneth                                           |
| <b>CTC:</b> i’d recommend him to you instead of blackstone thanks laupped kenneth                                            |
| <b>LLM (NB):</b> i d e r c o m e n d h i m t o y o u i n s t e a d o f b a c k s t o n e t h a n s l a l p e d k e n n e t h |
| <b>Stage 2 – Failure: Formatting / Translation</b>                                                                           |
| <b>Real:</b> if they tried to run they were hit from behind if they stood still they were clubbed carefully                  |
| <b>CTC:</b> if they tried to run they were hit from behind if they stood still they were clubbed carefully                   |
| <b>LLM (WB):</b> if they tried to run they were hit from behind if they stood still they were clubbed carefully              |
| <b>LLM (NB):</b> iftheytriedtoruntheywerehitfrombehindiftheystoodstilltheywereclubbedcarefully                               |
| <b>Real:</b> oh papa by that testing everything by the standard of wealth                                                    |
| <b>CTC:</b> oh papa by that testing everything by the standard of wealth                                                     |
| <b>LLM (WB):</b> oh papa by that testing everything by the standard of wealth                                                |
| <b>LLM (NB):</b> o papá porque tudo é avaliado pelo padrão da riqueza                                                        |

## A.10 Effect of Restricted CTC Vocabulary as input to LLM

When SFM outputs are tokenized using the restricted vocabulary  $V_{ctc}$  and fed directly to the LLM (without Stage 2 fine-tuning), the LLM generates outputs at a different granularity than expected, most commonly producing character-level or sub-word-level output rather than the word level transcriptions seen during the LLM’s own pretraining. Table 9 quantifies this effect by measuring WER when ground-truth transcripts (rather than speech) are used as input, thereby isolating the impact of tokenizer mismatch from acoustic modeling errors. The results reveal two notable patterns. First, Phi-3.1-3.8B is considerably more sensitive to vocabulary mismatch than Qwen3-4B. Second,

no clear scaling trend is observed across Qwen3 model sizes, suggesting that the issue is not simply a function of model capacity but rather of how well each LLM accommodates restricted vocabulary tokenization. Importantly, the elevated WER values reported under  $V_{\text{ctc}}$  in Table 9 are attributable primarily to formatting errors (character-level outputs, missing spaces, occasional translation) rather than genuine transcription errors. These errors are resolved after Stage 2 finetuning, as shown in the main results.

Table 9: Effect of using a  $\sim 1\text{k}$  subset of the LLM vocabulary for tokenizing SFM-OES. When evaluated on LibriSpeech *test-other* using ground-truth transcripts as input to the ASR prompt. Performance degrades substantially under  $V_{\text{ctc}}$ , with Phi-3.1-3.8B showing higher sensitivity than Qwen-3-4B. No clear scaling trend is observed across Qwen models. The reader should keep in mind that inflated WER is because of formatting errors as shown in Table 8 in the Appendix. Furthermore, these errors to a large extent are resolved after finetuning.

| Model                                                 | Vocab            | WER (%) |
|-------------------------------------------------------|------------------|---------|
| <i>Base comparison</i>                                |                  |         |
| Phi-3.1-3.8B                                          | $V_{\text{llm}}$ | 0.91    |
| Qwen-3-4B                                             |                  | 0.02    |
| Phi-3.1-3.8B                                          | $V_{\text{ctc}}$ | 6.91    |
| Qwen-3-4B                                             |                  | 4.40    |
| <i>Qwen scaling under <math>V_{\text{ctc}}</math></i> |                  |         |
| Qwen-3-0.6B                                           | $V_{\text{ctc}}$ | 6.46    |
| Qwen-3-1.7B                                           |                  | 4.89    |
| Qwen-3-8B                                             |                  | 6.33    |
| Qwen-3-14B                                            |                  | 3.03    |
| Qwen-3-32B                                            |                  | 10.37   |